

Reinforcement Learning with Verifiable Rewards for Agentic Language Models

Markus Junttila

December 2025

Contents

1	Introduction	2
2	State of RLVR	2
2.1	The Group-Based Baseline (GRPO)	2
2.2	Loss Type	3
2.3	Advantage	4
2.4	Loss Aggregation	4
2.5	KL Penalty	5
2.6	Entropy Collapse	6
2.7	Clipping Mechanism	6
2.8	Dynamic/Active Sampling	6
2.9	Inference and Training Engine Mismatch	7
2.10	Multi-Turn Training	7
3	Methods	7
3.1	Environments and Model	7
3.2	Training Configuration	8
3.3	Ablation Study	12
4	Results	12
4.1	Comparative Performance	12
4.2	Training Dynamics	14
4.3	Ablation Study	17
5	Conclusions	20

1 Introduction

Large language models (LLMs) are transitioning from static, single-turn answer systems to agents capable of reasoning, navigating complex environments, and acting in multi-turn loops. This shift is primarily enabled by recent advances in reinforcement learning with verifiable rewards (RLVR); a training paradigm that leverages ground-truth outcomes rather than subjective human preferences. The approach demonstrated its power with the release of OpenAI o1 [1] and DeepSeek-R1 [2] models. Consequently, the era of relying solely on reinforcement learning from human feedback (RLHF) [3] with proximal policy optimization (PPO) [4] is ending, as the focus shifts towards RLVR.

However, the rapid expansion of research that started in late 2024 and early 2025 has fragmented the technical landscape. The field is now saturated with diverse criticless, group-based algorithms, such as GRPO, GSPO, and DAPO [5, 6, 7]. This proliferation of novel algorithms makes it difficult to understand the state of the field and raises adoption barriers for practitioners. In addition to novel ideas, there are conflicting methodologies regarding core implementation details, including advantage normalization [5, 8, 9], loss aggregation [5, 7], and the utility of the KL penalty [10, 7, 11, 12]. Notably, many of these approaches are completely orthogonal, allowing them to be combined into a single algorithm with improved effectiveness.

This report aims to address increasing complexities by providing a comprehensive snapshot of the field as of December 2025, alongside a comparative evaluation of selected algorithms in two complex environments, ALFWorld [13] and WebShop [14]. Section 2 examines different approaches to algorithm design and their impact on training dynamics. Sections 3 and 4 present the methodology and results of the empirical study.

2 State of RLVR

This section provides a detailed technical dissection of the current reinforcement learning with verifiable rewards (RLVR) landscape. The following subsections examine the evolution of objective function design, advantage estimation, KL regularization, entropy management, and sampling strategies, highlighting how recent algorithms like DAPO [7], Dr. GRPO [9], and Reinforce++ [8] address the fundamental challenges of reinforcement learning at scale. Where possible, these technical discussions are grounded by identifying the major model releases and algorithmic studies that have adopted or validated these specific modifications.

2.1 The Group-Based Baseline (GRPO)

The quick expansion of research into reinforcement learning with verifiable rewards started with group relative policy optimization (GRPO) [5]. This method shifted the paradigm from single-trajectory updates to group-based estimation, eliminating the need for a separate value network. The core objective function serves as the foundation for novel RLVR algorithms:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{x \sim P(X), \{y_i\} \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min(r_{i,t} A_{i,t}, \text{clip}(r_{i,t}, 1 - \varepsilon, 1 + \varepsilon) A_{i,t}) \right], \quad (1)$$

where $r_{i,t} = \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}$.

Here, for each prompt x , the old policy generates a group of G outputs $\{y_1, \dots, y_G\}$. The term $r_{i,t}$ represents the importance sampling ratio, ensuring that updates remain mathematically valid even as the current policy π_{θ} diverges from the data-collecting policy $\pi_{\theta_{\text{old}}}$ over multiple mini-batch updates. The clipping mechanism restricts the update size to a trust region. It acts as a pessimistic lower bound, ignoring the gradients when the probability ratio $r_{i,t}$ moves

outside the range $[1 - \varepsilon, 1 + \varepsilon]$ if doing so would improve the objective [4]. Often, the default value $\varepsilon = 0.2$ is used.

GRPO computes the advantage $A_{i,t}$ directly from the group statistics. For a given output y_i with reward R_i , the advantage is computed by normalizing the score against the rollout group’s mean and standard deviation:

$$A_{i,t} = \frac{R_i - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}. \quad (2)$$

This group-relative normalization incentivizes the model to generate outputs that are better than the group average.

While GRPO established a strong baseline for group-based algorithms, later research has identified limitations in its specific design choices. The remainder of this section analyzes the state of the field by deconstructing Equation 1 and examining how recent algorithms modify the components.

2.2 Loss Type

The PPO/GRPO-styled policy optimization objective has been the standard for post-training language models during the past few years due to its success in RLHF. During 2025, the surrogate objective, which clips the importance ratio $r_{i,t}$ to form a pessimistic lower bound, has been under scrutiny.

The authors of GSPO [6] argue that the standard GRPO [5] objective is ”ill-posed”, because the importance ratio does not serve its intended purpose in the equation. The principle of importance sampling is to estimate the expectation of a function under a target distribution by re-weighting samples from a behavior distribution. This correction requires averaging over multiple samples to be statistically valid. GRPO [5], however, applies the importance weight $r_{i,t}$ at each individual token position based on a single sample from the rollout distribution. In addition, the reward for the response is assigned at the sequence-level, and the authors argue that the unit of optimization objective should match the unit of the reward. Practically, they do this by re-defining the importance ratio to act at sequence-level instead of token-level as in GRPO [5], and formulating the objective as:

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E}_{x \sim P(X), \{y_i\} \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right) \right], \quad (3)$$

where the importance weight $s_i(\theta)$ is derived from the sequence likelihood:

$$s_i(\theta) = \left(\frac{\pi_{\theta}(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})} \right) \quad (4)$$

On the other hand, the team behind MiniMax.M1 [12] identified a practical failure mode in the GRPO objective [5], specifically, the clipping mechanism. This led to the development of the CISPO objective [12]. In the GRPO objective, when the importance ratio $r_{i,t}$ falls outside the trust region defined by the ε parameter, the gradient related to the token in question is zeroed. MiniMax researchers observed that tokens associated with reflective behaviors (e.g., ”wait,” ”however,” ”aha”) are often clipped, and these ”forks” of reasoning are never reinforced. This phenomenon is amplified with the training setup of MiniMax-M1, which involves 16 rounds of off-policy updates per generation batch [12]. To address this, CISPO implements clipped importance sampling objective:

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{j=1}^G |y_j|} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \text{sg}(\hat{r}_{i,t}) A_{i,t} \log \pi_{\theta}(y_{i,t} \mid x, y_{i,<t}) \right], \quad (5)$$

where $\hat{r}_{i,t}$ is the clipped importance sampling weight

$$\hat{r}_{i,t} = \text{clip}\left(\frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}, 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}\right). \quad (6)$$

Without weight clipping CISPO reduces to the standard policy gradient objective. This objective allows the gradients to flow through the log probability term even when the model diverges significantly from the reference, while the importance sampling weight itself is clipped.

2.3 Advantage

There are two main components of the advantage term that have been scrutinized by the recent research. The computation of the baseline and the baseline normalization strategy. The GRPO-style advantage baseline computation, seen in Equation 2, has been adopted in the majority of significant releases, including Magistral, MiniMax-M1, GLM-4.5 and Olmo 3 [12, 15, 16, 11], but some alternatives exist.

For example, REINFORCE Leave-One-Out (RLOO) [17] samples G responses and calculates the baseline for sample y_i using the mean of the remaining $G - 1$ samples:

$$A_i^{\text{RLOO}} = R_i - \frac{1}{G-1} \sum_{j \neq i} R_j. \quad (7)$$

Although RLOO offers a theoretically unbiased estimator, when the group size G grows larger, the difference compared to the GRPO-baseline diminishes.

More research has been done in the area of advantage normalization strategies. GRPO-baseline normalizes the advantage with the reward group’s (local) standard deviation. This approach may cause issues with highly concentrated reward distributions (e.g., $\{0, 1\}$), as often all the rollouts in a reward group achieve the same reward and the standard deviation $\text{std}(\mathbf{R})$ becomes zero. This may result in exploding gradients and numerical instability. To mitigate this, distinct normalization strategies have emerged.

Reinforce++ [8] proposes to normalize the advantage with batch (global) standard deviation across all rollouts:

$$A_i^{\text{Batch}} = \frac{R_i - \text{mean}(\mathbf{R}_{\text{batch}})}{\text{std}(\mathbf{R}_{\text{batch}})}. \quad (8)$$

The mean can be chosen to be taken within the whole batch ($\text{mean}(\mathbf{R}_{\text{batch}})$) to obtain the standard Reinforce++ algorithm, or the rollout group ($\text{mean}(\mathbf{R})$) to obtain a Reinforce++-Baseline variant of the algorithm. Normalizing by the batch standard deviation alleviates the issues with zero standard deviation, as the variation within a batch of rollouts is typically larger than the variation within a group. Using this method, an improvement over the group-level normalization has been reported by algorithmic studies [18, 19] and adopted in model releases [15].

Dr. GRPO [9] proposes to omit the standard deviation term altogether to solve the same issue:

$$A_i = R_i - \text{mean}(\mathbf{R}). \quad (9)$$

This approach has also been adopted in large-scale model releases [11].

2.4 Loss Aggregation

The GRPO implementation for loss aggregation strategy is so-called sample-average, where each token’s contribution to the loss is first averaged within one rollout sequence of a reward group and then each rollout’s contributions are averaged within the reward group G [5]:

$$\mathcal{L}_{\text{Sample}} = \frac{1}{G} \sum_{i=1}^G \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} L_{i,t} \right) \quad (10)$$

This aggregation strategy ensures that each reward group contributes to the loss equally, but it introduces a response-level length bias.

First acknowledged in the Dr. GRPO paper [9], the division by $|y_i|$ before averaging between reward groups creates imbalance in how the model perceives the reward based on the length of individual rollouts. The magnitude of gradient that flows through each individual token is scaled by $\frac{1}{|y_i|}$, so shorter sequences have higher individual gradient magnitudes. This incentivizes the model to produce shorter responses when it is more certain of the answer and longer responses when the variance is larger. To combat this phenomenon, Dr. GRPO [9] proposes to leave out the normalization by $|y_i|$:

$$\mathcal{L}_{\text{Dr. GRPO}} = \frac{1}{G} \sum_{i=1}^G \left(\sum_{t=1}^{|y_i|} L_{i,t} \right) \quad (11)$$

By removing the response length, the total gradient magnitude becomes proportional to the sequence length.

Another variant of loss aggregation was proposed along the DAPO algorithm [7], that proposes a token average strategy within a batch. This method sums up token-level losses of each response across all reward groups, and then divides the loss by the number of tokens in all generations:

$$\mathcal{L}_{\text{DAPO}} = \frac{1}{\sum_{j=1}^G |y_j|} \sum_{i=1}^G \sum_{t=1}^{|y_i|} L_{i,t} \quad (12)$$

This method ensures that each token contributes equally to the loss, regardless of the length of the sequence that the token belongs to. This token-level loss aggregation has been adopted in several recent model releases [12, 15, 11].

2.5 KL Penalty

The Kullback-Leibler (KL) divergence penalty is a core feature of the PPO-based RLHF paradigm. In the RLHF scenario it is thought to be important to align the model’s behavior without diverging too far from the initial model. The penalty prevents the current policy π_θ from deviating too far from the initial reference policy π_{ref} by assigning a penalty proportional to KL divergence estimate between the policies to either reward term

$$R'_{i,t} = R_{i,t} - \beta \log \left(\frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\text{ref}}(y_{i,t} \mid x, y_{i,<t})} \right) \quad (13)$$

or objective term

$$\mathcal{J}_{\text{total}}(\theta) = \mathcal{J}_{\text{RL}}(\theta) - \beta \mathbb{E}_x [D_{\text{KL}}(\pi_\theta(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))]. \quad (14)$$

There are different methods and mathematical forms to estimate the KL divergence itself, and more information on the topic can be found in [20] and [8].

The need to restrict the policy from diverging from the reference policy has been under scrutiny, as in an RLVR setting deviation from initial policy is often desirable if that helps the policy achieve correct answers on the verifiable tasks. Recent algorithmic studies [7, 9], as well as model releases [12, 16, 11] have concluded that KL regularization is not necessary in RLVR, as it hinders performance and does not lead to over-optimization.

Despite the trend toward removal of the penalty, for example, recent study by Nvidia [10] suggests that KL divergence penalty has a role in mitigating entropy collapse (see Section 2.6) of the model. The study argues that while tricks like clip-higher (see Section 2.7) and high rollout temperature can help slow entropy collapse, explicit regularization via KL penalty provides a stronger and more stable solution.

2.6 Entropy Collapse

Traditionally, maintaining high policy entropy is regarded as essential for ensuring sufficient exploration by the model and preventing premature convergence to suboptimal local minima. However, it has been noticed that GRPO-style optimization suffers from entropy collapse: the entropy of the policy decreases quickly as training progresses [7, 10, 21, 22]. The sampled responses for certain tasks can become nearly identical in a reward group, making it difficult to assign relative advantages.

Recent theoretical analysis shows that the change in the policy entropy is driven only by the unclipped tokens [21, 22]. These are the tokens where the importance ratio falls inside the trust region defined by the clipping bounds in Equation 1. For the clipped tokens, where the importance ratio falls outside of the trust region, the gradients are effectively zeroed because of the min operation in the objective. The direction of policy entropy change is determined by the covariance between the token’s log probabilities and their estimated advantages [21, 22]. Specifically, this means that updating the policy with tokens that have high probability and positive advantage, or low probability and negative advantage, sharpens the distribution and lowers the entropy. Conversely, updating the policy with high probability and negative advantage, or low probability and positive advantage tokens, smooths the distribution and increases entropy. This ties back to optimization objective (see Equation 1) such that, empirically, tokens with low probability tend to have either very high or very low importance sampling weights and are therefore clipped [21]. In the standard implementation of algorithms with symmetric clipping bounds (see Equation 1), a majority of positive, low probability tokens are prevented from contributing to the optimization. This systematic exclusion of entropy increasing tokens causes the generation entropy to decline during the training [21].

Recognizing these dynamics, some research has shifted towards entropy recognizing objectives. BAPO [21] dynamically adjusts the clipping ranges until the target proportion of contribution from positive signals to the policy gradient loss is met. Clip-Cov [22] clips a small fraction of high-covariance tokens out from the policy gradient updates and KL-Cov [22] applies KL penalty to a fraction of high-covariance tokens. At the time of this study, these insights have not been adopted in large-scale model releases.

2.7 Clipping Mechanism

To address the entropy collapse issue described in Section 2.6, DAPO [7] proposes to decouple the lower and upper clipping bounds of Equation 1. Specifically, they propose assigning a higher threshold to the upper bound (e.g., $\varepsilon_{\text{upper}} = 0.28$) while maintaining a conservative lower bound ($\varepsilon_{\text{lower}} = 0.2$). This modification facilitates the generation of more diverse samples and keeps the policy’s entropy from collapsing. The effectiveness of the method can be linked to the theoretical discussion of Section 2.6. The upper threshold only affects tokens with positive advantages, which means that clip-higher mechanism is adding more low probability and high advantage tokens to the policy gradient updates, which increases the policy entropy. The method has been validated in algorithmic studies [19], [18], and has been adopted in recent model releases [11].

2.8 Dynamic/Active Sampling

If every rollout within a reward group G receives the same reward from the environment, the normalized advantages become zero, and the resulting gradient update is nullified. Empirically, as training continues, the number of rollout groups where every reward is the same (e.g., reward 1 for success) increases. This in turn decreases the number of effective prompts that are processed in each training step. To solve this, DAPO [7] proposes to over-sample and filter out prompts where the reward is identical for every rollout (e.g., 0 or 1), leaving all prompts

in the batch with effective gradients. A refined method is adopted in the Olmo 3 recipe [11], where an asynchronous framework continuously pulls completions until a target batch size of non-zero gradient samples is reached. A similar effect may be achieved with curriculum learning strategies.

2.9 Inference and Training Engine Mismatch

Modern RL frameworks (e.g., VeRL [23], OpenRLHF [24]) optimize throughput by utilizing an inference engine (e.g., vLLM [25], SGLang [26]) for data collection, and a training engine (e.g., FSDP [27], Megatron [28]) for gradient computation. Despite the model weights being the same, due to deviating implementations of the engines, an unexpected rollout-training mismatch is observed: the produced token probabilities may be significantly different [29, 30]. Instead of hurting the throughput speed considerably and forcing the engines to produce identical output, recent work proposes to embrace the mismatch using an algorithmic correction [29]. Truncated importance sampling (TIS) treats the inference engine as a separate behavior policy and corrects the distribution shift by using an additional importance ratio. For GRPO-style objective, the TIS implementation results in the following objective:

$$\mathcal{J}_{\text{TIS}}(\theta) = \mathbb{E}_{\{y_i\} \sim \pi_{\text{inf}}} \left[\frac{1}{\sum_{j=1}^G |y_j|} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \min \left(\frac{\pi_{\text{train}}(y_{i,t}, \theta_{\text{old}})}{\pi_{\text{inf}}(y_{i,t}, \theta_{\text{old}})}, C \right) \cdot \min(r_{i,t} A_{i,t}, \text{clip}(r_{i,t}, 1 - \varepsilon, 1 + \varepsilon) A_{i,t}) \right], \quad (15)$$

where a truncated importance ratio between the training and inference probabilities is introduced. C is a hyperparameter in TIS. Importance sampling is used to correct the training-inference mismatch in recent model releases, like Olmo 3 [11] and NVIDIA Nemotron 3 [31].

2.10 Multi-Turn Training

An area of research has focused on algorithms that better support the LLM paradigm shift from single-turn answering to agentic problem solving. In long-horizon environments standard GRPO [5] struggles with the credit assignment, as the reward is typically sparse and given only at the end of a trajectory. The algorithm assigns the same advantage score to every step in the sequence, making it difficult to distinguish pivotal decisions from irrelevant ones. GiGPO [32], addresses this by introducing a step-level advantage in addition to the episode-level advantage. An anchor-state-grouping mechanism identifies identical environment states that occur in different rollouts. GiGPO groups actions from the shared environment states and computes a relative step-level advantage by comparing the discounted future returns of different actions taken from the exact same context. The step-level advantage is linearly combined with the standard episode level advantage, and the combined value is used in a standard GRPO-style optimization objective (see Equation 1).

3 Methods

To empirically evaluate the theoretical aspects of algorithms discussed in Section 2, a comparative study of six RLVR algorithms in agentic, multi-turn environments was conducted. This section goes through the experimental setup, training configurations, and ablation strategies.

3.1 Environments and Model

For all experiments, the Qwen2.5-1.5B-Instruct [33] model was utilized as the policy model. This size was chosen to balance reasoning capability with computational efficiency, allowing for full fine-tuning without the need for parameter-efficient approximations like LoRA or QLoRA.

The same model is also used in other similar studies [32, 34, 35, 36], which makes comparing the results with existing literature easier.

Two text-based environments, ALFWorld [13] and WebShop [14], were selected for the study. This was done because the used model is an LLM, and these environments require innate world knowledge and multi-turn reasoning capabilities, providing a challenging platform for algorithm evaluation and comparison. The environments allow for multi-turn (agentic) training, where the model interacts with the environment to solve the task at hand. This differs from the traditional single-step training paradigm that is widely adopted for LLMs. In both environments, the model first outputs thinking tokens within `<think></think>` tags and then the action within `<action></action>` tags.

ALFWorld [13] is a simulated, embodied, household environment, where the model navigates the environment to solve common household tasks using natural language instructions. The tasks are sampled from six task classes (e.g., "pick and place item", "pick and clean item"), and the tasks are to be completed using 12 different actions (e.g., "go to", "pick", "put"). The environment contains 120 distinct rooms (30 kitchen, 30 bedrooms, 30 bathrooms and 30 living rooms). The training task dataset contains 3,553 unique tasks. The performance of the model is evaluated with two evaluation task datasets. In-distribution dataset (140 tasks) contains new tasks, but the rooms are the same as in the training set. Out-of-distribution dataset (134 tasks), on the other hand, contains new tasks, and the tasks are completed in rooms previously unknown to the model.

WebShop [14] is a simulated e-commerce website environment, where the goal is to search and purchase items using natural language instructions. The environment contains 12,087 unique goals and 1.18 million items to select from. The two actions available to the model are "search", where model inputs a natural language query for the search engine used inside the environment and "click", where model clicks buttons to select items, pick options, and finally purchase the item. The training dataset is made up of 11,587 goals and the evaluation dataset 500 goals. The WebShop environment computes a score in range $[0, 1]$ that shows how "close" the item purchased by the model is to the item for which the text-based instructions were created. The score is computed by matching the attributes, options and price requirement of the picked item and the goal item. In addition to the WebShop score metric, the success rate is also tracked. The success rate is defined by the model scoring the maximum WebShop score of 1, so all attributes, options and the price requirement of the picked item match the goal item.

3.2 Training Configuration

The training was conducted using `verl-agent` [32] library. The library is a fork of `VeRL` [23] library and it was used because `VeRL` didn't allow multi-turn training during this algorithm comparison study. To ensure fair comparison, all algorithms were implemented in their most basic form, using recommended hyperparameters, without additional parameter tuning. The comparative study evaluates the following six algorithms: GRPO, DAPO, GSPO, GiGPO, RLOO, and Reinforce++ [5, 7, 6, 32, 17, 8]. The algorithms were selected based on diversity and popularity during the study. KL penalty is used in every training run of the main comparative study. The main training parameters used in ALFWorld and WebShop environments are displayed in Tables 1 and 2. More specific algorithm-wise parameters can be seen in training launch scripts. One training run for the same number of training steps is done for each algorithm in both environments.

More robust results would be achieved if multiple training runs were performed for each algorithm and the results were averaged. This was unfeasible for this study due to computational and time constraints. To validate using just one training run for each algorithm, a small-scale stability analysis was performed with GRPO in the ALFWorld environment. A model was trained with GRPO with otherwise identical training setup, but random seed (affects the sampled tasks) was varied between runs. The training dynamics of the runs can be seen in

Figure 1. The training dynamics exhibited consistent patterns and performance seemed to approximately converge, justifying the single-run approach for the broader comparative study.

Table 1: Hyperparameters for the ALFWorld environment training.

Hyperparameter	Value
<i>General Setup</i>	
Foundation Model	Qwen2.5-1.5B-Instruct
Total RL steps	300
Max Prompt Length	2048
Max Response Length	512
Test Frequency (steps)	5
Max environment steps	50
<i>Optimizer & Reward</i>	
Optimizer	AdamW
Learning Rate	1×10^{-6}
LR Schedule	Constant
KL Coefficient	0.01
Reward success/failure	10.0/0.0
Invalid Action Penalty	0.1
Temperature rollout/test	1.0/0.4
<i>Batch Sizes & Compute</i>	
Train Batch Size	16
Rollout Group Size	8
Mini-batch Size	256
Micro-batch Size	32
Number of GPUs (H100)	2

Table 2: Hyperparameters for the WebShop environment training.

Hyperparameter	Value
<i>General Setup</i>	
Foundation Model	Qwen2.5-1.5B-Instruct
Total RL steps	600
Max Prompt Length	8192
Max Response Length	1024
Test Frequency (steps)	5
Max environment steps	15
<i>Optimizer & Reward</i>	
Optimizer	AdamW
Learning Rate	1×10^{-6}
LR Schedule	Constant
KL Coefficient	0.01
Reward	$[0, 1]$ (+0.2 if success)
Invalid Action Penalty	0.05
Temperature rollout/test	1.0/0.4
<i>Batch Sizes & Compute</i>	
Train Batch Size	4
Rollout Group Size	8
Mini-batch Size	8
Micro-batch Size	2
Number of GPUs (H100)	2

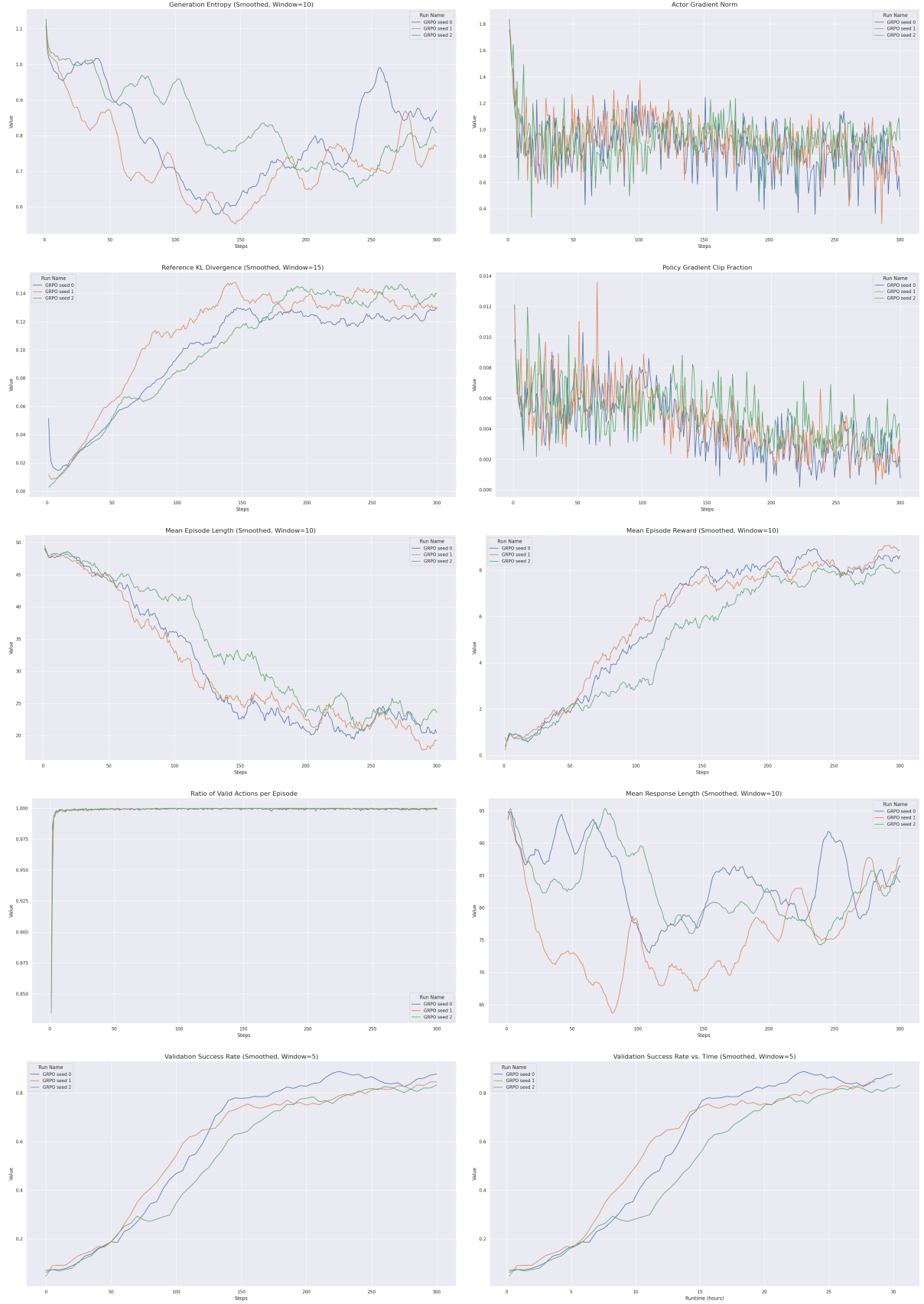


Figure 1: Training dynamics of GRPO in ALFWorld, when varying the random seed. Random seed affects the tasks that are sampled from the training dataset. Some plots are smoothed using a running average over a certain window.

3.3 Ablation Study

In addition to the algorithm comparison, a targeted ablation study was performed on the GRPO baseline within the ALFWorld environment to isolate the impact of two major design choices: KL penalty (see Section 2.5) and clip-higher mechanism (see Section 2.7). As discussed previously, using KL penalty in RLVR training has been under heavy scrutiny, and this ablation seeks to reveal if this critique is justified. The clip-higher mechanism’s main function is to maintain the exploration of the model by keeping the generation entropy (see Section 2.6) consistent through the training. Mainly, the aim is to keep the entropy from collapsing during the training.

4 Results

The results of the empirical comparative study and the ablations are presented in this section. The results focus on the final task performance of the models trained with different algorithms, and the training dynamics.

4.1 Comparative Performance

As visualized in Figures 2 and 3, and detailed in Table 3, DAPO and GiGPO emerge as the top-performing algorithms, consistently outperforming the GRPO baseline in both environments. In ALFWorld, DAPO achieves a 97.0% in-distribution (ID) and 89.0% out-of-distribution (OOD) success rate, closely followed by GiGPO. GSPO underperforms in this environment compared to other algorithms with 63.9% ID success rate. In WebShop, DAPO again achieves the highest performance with a 33.8% success rate and a 70.4 score. RLOO has the worst results, only achieving 11.4% success rate and 30.3 score. Besides DAPO and GiGPO, other algorithms don’t show consistent improvement upon the baseline algorithm GRPO, that achieves 84.9% ID and 80.7% OOD success rates in ALFWorld, and 20.3% success rate and 51.8 score in WebShop.

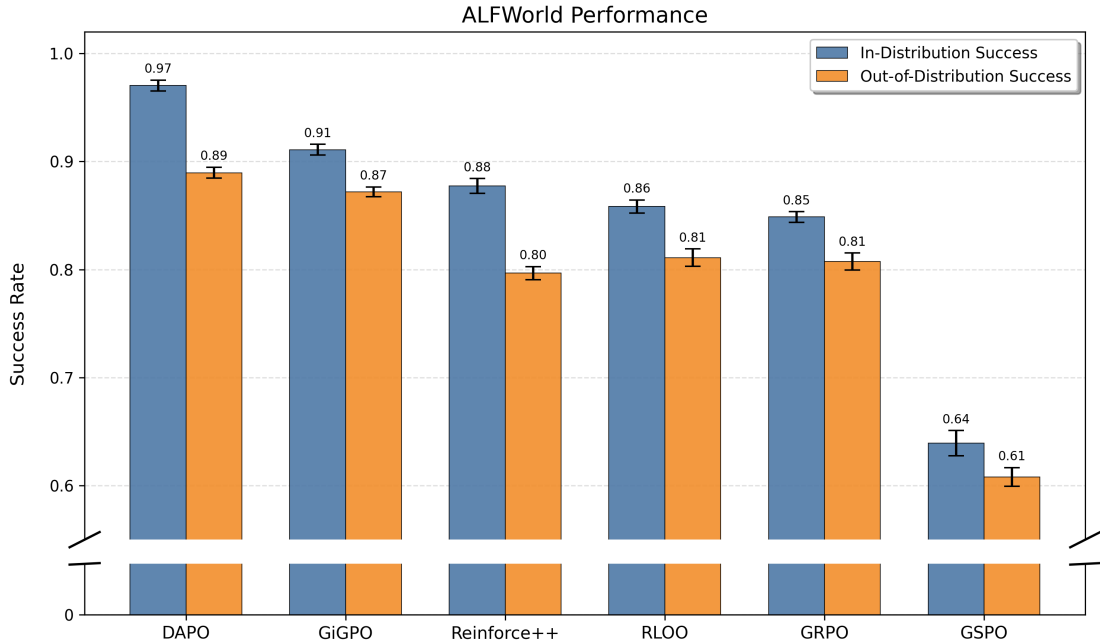


Figure 2: ALFWorld results

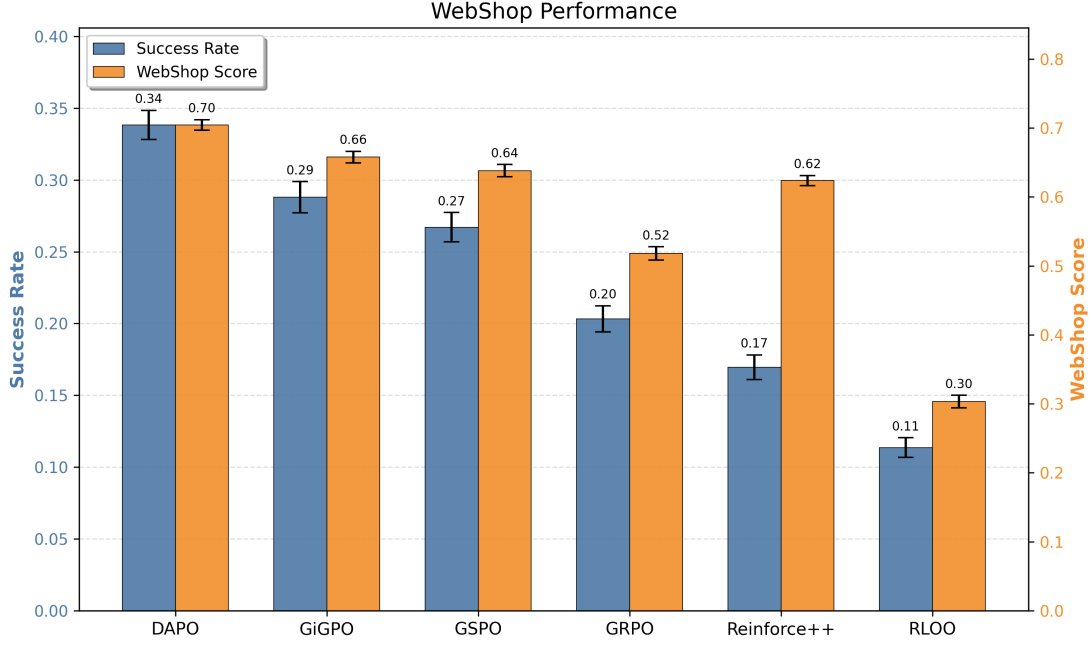


Figure 3: WebShop results

Table 3: Main results. We report the ID and OOD success rates for ALFWorld tasks and the success rate and the task score for WebShop. Results are formatted as $\text{Mean} \pm \text{Std}$. Statistics are computed across repeated evaluations of each trained model: 30 per split with 128 sampled tasks per evaluation for ALFWorld, and 200 with 32 sampled tasks per evaluation for WebShop.

Method	Split	ALFWorld Success Rate (%)							WebShop	
		Pick	Clean	Heat	Cool	Look	Pick 2	Total	Score	Success
Reinforce++	ID	90.5 $\pm$ 1.6	95.5 $\pm$ 1.4	99.0 $\pm$ 0.9	82.5 $\pm$ 0.6	78.1 $\pm$ 2.5	83.3 $\pm$ 1.9	87.7 $\pm$ 0.7	62.4 $\pm$ 0.7	17.0 $\pm$ 0.9
	OOD	63.3 $\pm$ 1.6	75.3 $\pm$ 1.7	84.6 $\pm$ 0.9	83.3 $\pm$ 1.0	100.0 $\pm$ 0.0	92.7 $\pm$ 1.7	79.7 $\pm$ 0.6		
RLOO	ID	86.5 $\pm$ 1.1	96.8 $\pm$ 1.5	99.0 $\pm$ 1.0	68.6 $\pm$ 1.7	97.5 $\pm$ 1.5	82.5 $\pm$ 1.1	85.8 $\pm$ 0.6	30.3 $\pm$ 0.9	11.4 $\pm$ 0.7
	OOD	68.5 $\pm$ 1.2	72.9 $\pm$ 1.6	87.5 $\pm$ 2.2	94.9 $\pm$ 1.1	96.8 $\pm$ 2.0	69.8 $\pm$ 3.6	81.1 $\pm$ 0.8		
GRPO	ID	92.7 $\pm$ 1.5	97.6 $\pm$ 1.0	99.5 $\pm$ 0.7	80.8 $\pm$ 0.9	81.7 $\pm$ 2.7	66.0 $\pm$ 1.8	84.9 $\pm$ 0.5	51.8 $\pm$ 1.0	20.3 $\pm$ 0.6
	OOD	68.0 $\pm$ 1.7	78.6 $\pm$ 1.2	85.3 $\pm$ 2.0	94.0 $\pm$ 1.2	82.0 $\pm$ 3.5	73.6 $\pm$ 4.1	80.7 $\pm$ 0.8		
GSPO	ID	77.2 $\pm$ 1.4	70.3 $\pm$ 1.9	62.3 $\pm$ 3.3	53.1 $\pm$ 2.3	61.0 $\pm$ 3.8	59.7 $\pm$ 2.2	63.9 $\pm$ 1.2	63.8 $\pm$ 0.9	26.7 $\pm$ 1.0
	OOD	64.8 $\pm$ 1.3	48.1 $\pm$ 3.0	65.5 $\pm$ 2.3	60.9 $\pm$ 2.1	80.3 $\pm$ 3.8	57.5 $\pm$ 4.4	60.8 $\pm$ 0.9		
GiGPO	ID	95.1 $\pm$ 1.4	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	96.6 $\pm$ 0.6	78.0 $\pm$ 2.7	76.5 $\pm$ 1.5	91.1 $\pm$ 0.5	65.8 $\pm$ 0.8	28.8 $\pm$ 1.1
	OOD	66.7 $\pm$ 1.6	79.1 $\pm$ 1.2	93.7 $\pm$ 0.8	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	87.2 $\pm$ 0.5		
DAPO	ID	100.0 $\pm$ 0.0	99.5 $\pm$ 0.5	100.0 $\pm$ 0.0	96.1 $\pm$ 1.2	97.1 $\pm$ 1.7	91.8 $\pm$ 1.5	97.0 $\pm$ 0.5	70.4 $\pm$ 0.8	33.8 $\pm$ 1.0
	OOD	81.3 $\pm$ 1.0	93.6 $\pm$ 1.5	92.1 $\pm$ 1.3	99.0 $\pm$ 0.6	86.1 $\pm$ 3.5	65.8 $\pm$ 4.1	89.0 $\pm$ 0.5		

4.2 Training Dynamics

The training dynamics of algorithms in ALFWorld and WebShop environments are illustrated in Figures 4 and 5. The training dynamics for DAPO should be interpreted with the caveat that trajectories with identical results are filtered out during training. Additional rollouts during the sampling phase of RL in DAPO cause the training time to increase significantly with some training setups. ALFWorld has a binary reward distribution, and the training time of DAPO is nearly triple that of the other algorithms. In WebShop with non-binary reward distribution, the training time of DAPO is not significantly longer than that of the other algorithms.

DAPO and GSPO maintain higher generation entropy throughout the whole training in ALFWorld and during the early stages of training in WebShop. For other algorithms, the generation entropy profiles are more unpredictable, fluctuating significantly between the two environments. More consistent behavior of DAPO and GSPO is probably caused by DAPO’s clip-higher mechanism combined with dynamic sampling, and GSPO’s sequence-level loss formulation. However, consistent with findings of recent studies [10, 21], the entropy for even these methods (DAPO and GSPO) eventually begins to decline in prolonged training (WebShop), suggesting that while these mechanisms mitigate the entropy collapse, they do not fully prevent it.

GiGPO exhibits an efficiency advantage over other algorithms, achieving the lowest average episode lengths across both benchmarks. This could be driven by its step-level advantage estimation, which allows the model to optimize intermediate reasoning steps directly, rather than relying solely on the sparse, delayed reward at the end of the trajectory.

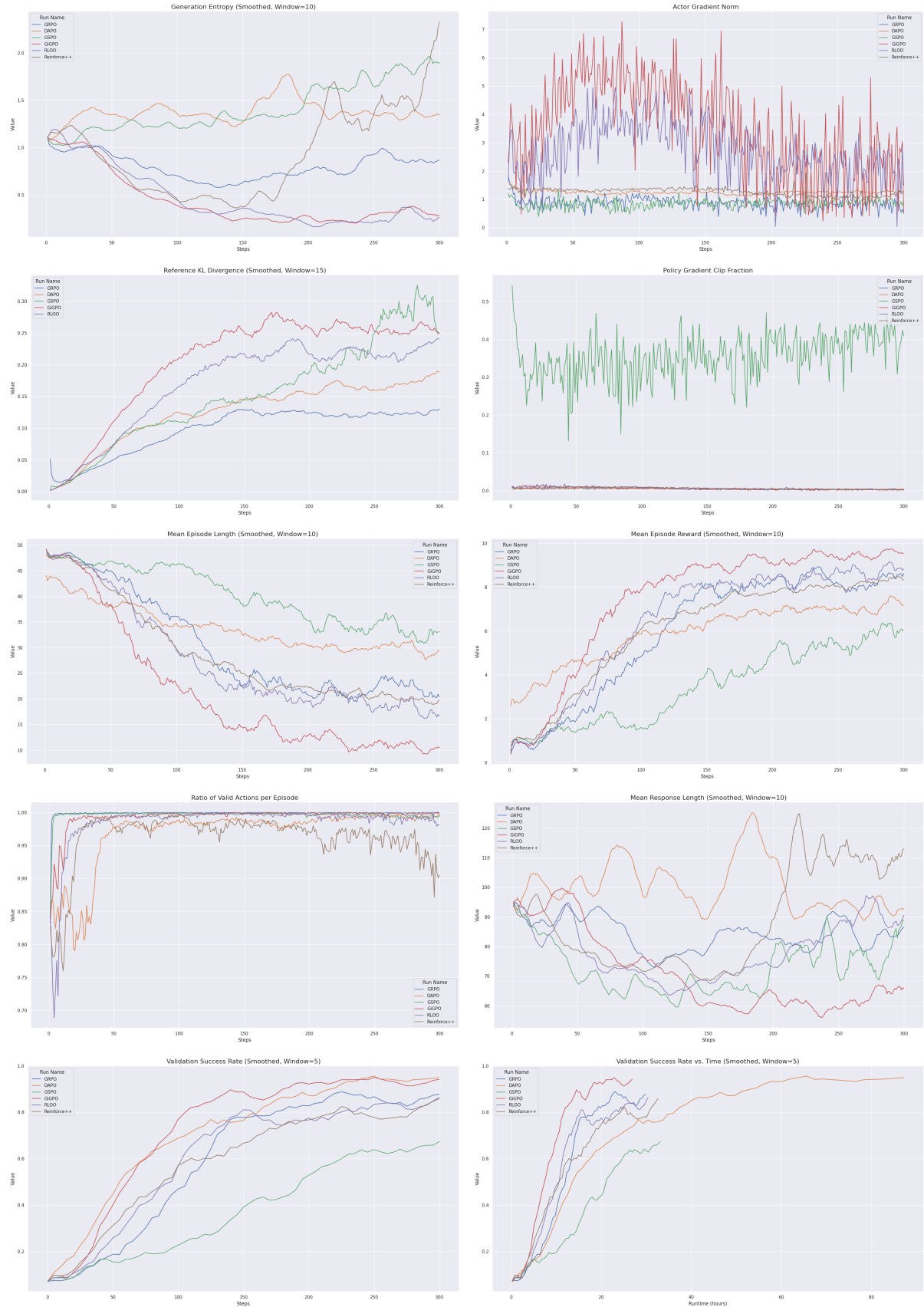


Figure 4: ALFWorld training dynamics. Some plots are smoothed using a running average over a certain window.

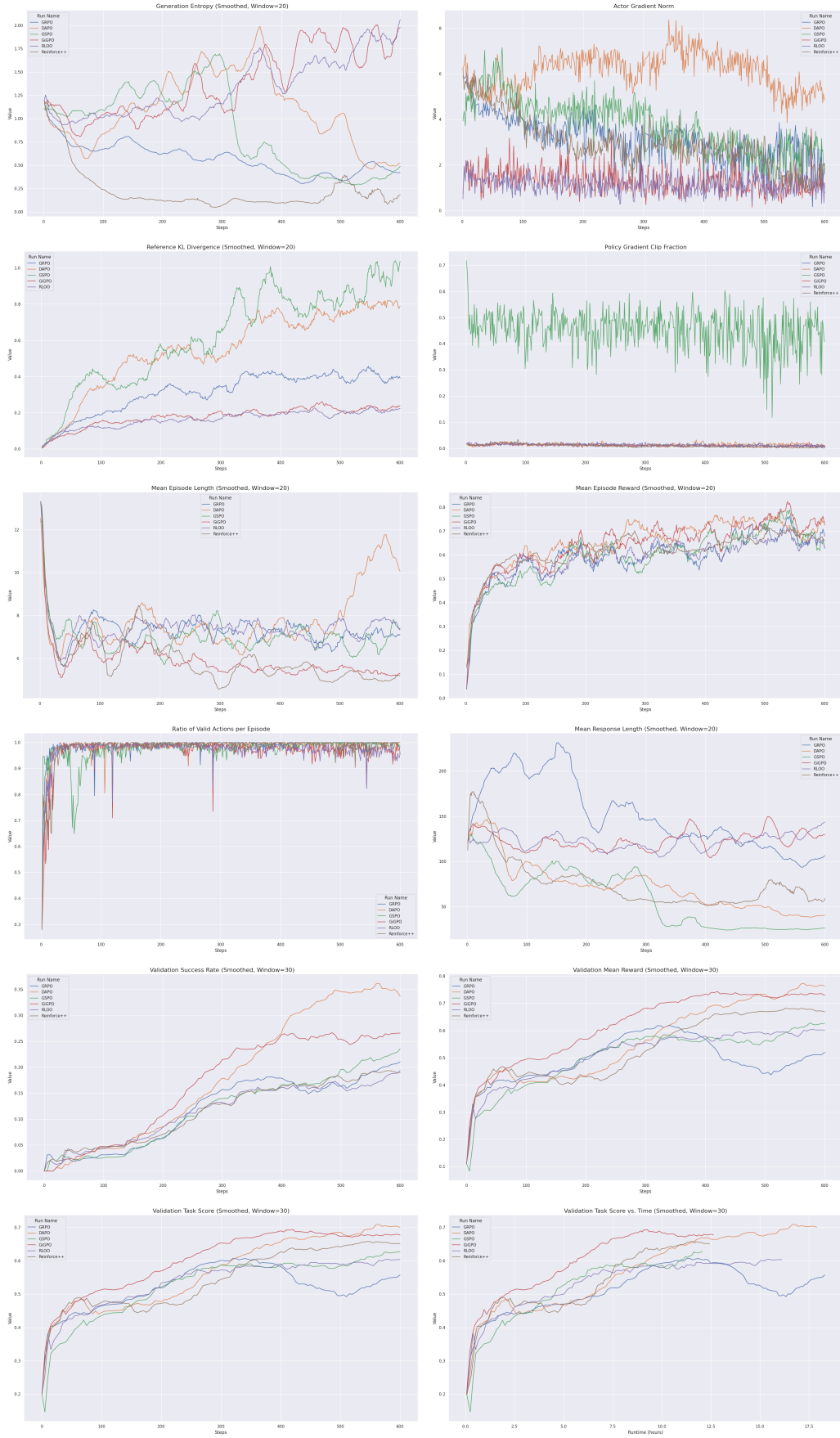


Figure 5: WebShop training dynamics. Some plots are smoothed using a running average over a certain window.

4.3 Ablation Study

The ablation results on the GRPO baseline are visualized in Figure 6, and detailed in Table 4. It can be seen that removing KL penalty consistently improves the performance in the ALFWorld environment. The clip-higher mechanism doesn’t consistently improve the final performance in the ablation study.

The ablation dynamics are presented in Figure 7. The entropy collapse phenomenon is observed especially with the no KL penalty and no clip-higher run. In line with ProRL study [10], adding KL penalty to GRPO increases the generation entropy and helps with the entropy collapse problem. The clip-higher mechanism functions as intended and further increases the generation entropy.

The configuration with the fewest restrictions, no KL penalty and clip-higher, results in more aggressive optimization and larger deviation from the initial policy. This run exhibits the largest KL divergence compared to the reference policy at the end of the training. One observable effect of the large deviation may be the significantly lower mean response length of the final model compared to other runs, which are more restricted through the use of KL penalty or stricter clipping bounds.

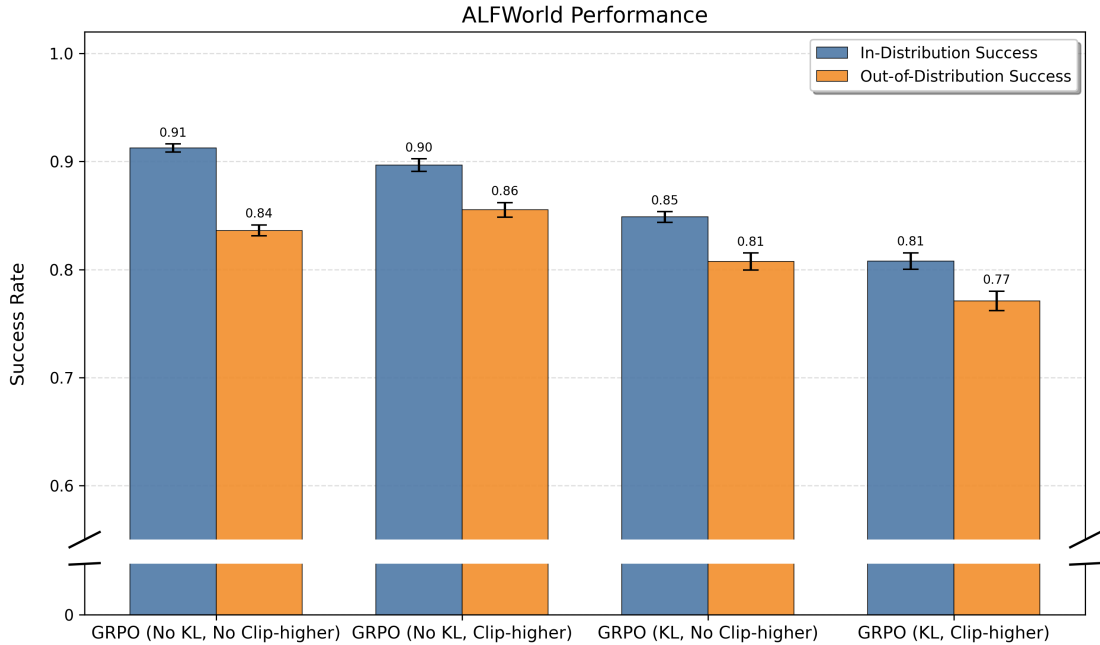


Figure 6: ALFWorld GRPO ablations results

Table 4: Ablation results. We report the ID and OOD success rates for ALFWorld tasks for different GRPO configurations. Results are formatted as $\text{Mean}_{\pm\text{Std}}^{\pm 95\% \text{ CI}}$. Statistics are computed across 30 repeated evaluations of each trained model per split, with 128 sampled tasks per evaluation.

Method	Split	ALFWorld Success Rate (%)						
		Pick	Clean	Heat	Cool	Look	Pick 2	Total
GRPO (KL, No clip-higher)	ID	92.7 ± 1.5 ± 4.3	97.6 ± 1.0 ± 2.9	99.5 ± 0.7 ± 2.0	80.8 ± 0.9 ± 2.5	81.7 ± 2.7 ± 7.4	66.0 ± 1.8 ± 5.0	84.9 ± 0.5 ± 1.4
	OOD	68.0 ± 1.7 ± 4.9	78.6 ± 1.2 ± 3.5	85.3 ± 2.0 ± 5.4	94.0 ± 1.2 ± 3.4	82.0 ± 3.5 ± 9.7	73.6 ± 4.1 ± 11.4	80.7 ± 0.8 ± 2.2
GRPO (No KL, No clip-higher)	ID	94.9 ± 0.8 ± 2.2	99.4 ± 0.6 ± 1.5	96.4 ± 1.6 ± 4.4	85.8 ± 0.9 ± 2.4	85.5 ± 2.6 ± 7.3	87.2 ± 1.2 ± 3.4	91.3 ± 0.4 ± 1.1
	OOD	65.5 ± 1.3 ± 3.8	82.5 ± 1.2 ± 3.3	85.7 ± 1.7 ± 4.7	92.2 ± 0.7 ± 2.0	95.5 ± 2.0 ± 5.5	92.3 ± 2.9 ± 8.1	83.6 ± 0.5 ± 1.4
GRPO (No KL, clip-higher)	ID	91.6 ± 1.5 ± 4.1	95.2 ± 1.1 ± 3.0	96.3 ± 1.8 ± 5.0	81.9 ± 1.8 ± 5.1	97.6 ± 1.5 ± 4.2	84.5 ± 1.3 ± 3.5	89.7 ± 0.6 ± 1.7
	OOD	76.1 ± 1.3 ± 3.6	87.5 ± 1.7 ± 4.6	89.1 ± 1.8 ± 5.1	88.6 ± 1.2 ± 3.3	98.2 ± 1.5 ± 4.3	76.6 ± 2.2 ± 6.2	85.5 ± 0.7 ± 1.9
GRPO (KL, clip-higher)	ID	85.5 ± 1.3 ± 3.7	90.4 ± 1.9 ± 5.3	98.4 ± 1.1 ± 3.2	78.7 ± 1.0 ± 2.9	79.5 ± 3.2 ± 9.0	62.9 ± 1.9 ± 5.4	80.8 ± 0.8 ± 2.1
	OOD	76.0 ± 1.4 ± 3.9	82.7 ± 1.7 ± 4.8	83.5 ± 2.0 ± 5.7	76.3 ± 1.9 ± 5.3	93.7 ± 2.2 ± 6.0	40.8 ± 3.7 ± 10.4	77.1 ± 0.9 ± 2.5



Figure 7: ALFWorld ablations dynamics. Some plots are smoothed using a running average over a certain window.

5 Conclusions

This study has provided a comprehensive snapshot of the rapidly evolving landscape of reinforcement learning with verifiable rewards as of late 2025, supported by a comparative benchmarking of six distinct algorithms and an ablation study isolating core design choices.

The comparative study demonstrated that the selection of a training algorithm is a decisive factor in the performance of reasoning agents. DAPO and GiGPO consistently outperformed the baseline, whereas methods such as GSPO and RLOO proved less robust to environmental variations. Furthermore, the ablation results indicate that for verifiable reasoning tasks, the KL penalty tends to hinder the performance by constraining the model to a suboptimal prior. The clip-higher mechanism works as intended, mitigating the entropy collapse, and maintaining a stable generation entropy throughout the training process.

Since this work implemented algorithms in their basic forms, future research could focus on optimizing performance by combining orthogonal methodologies of different algorithms. In particular, combining the best-performing algorithmic choices of this study, DAPO’s dynamic sampling and clip-higher mechanisms with GiGPO’s step-level advantage, could lead to improved results.

References

- [1] OpenAI, :, A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Hel-
yar, A. Madry, A. Beutel, A. Carney, A. Iftimie, A. Karpenko, A. T. Passos, A. Neitz,
A. Prokofiev, A. Wei, A. Tam, A. Bennett, A. Kumar, A. Saraiva, A. Vallone, A. Duber-
stein, A. Kondrich, A. Mishchenko, A. Applebaum, A. Jiang, A. Nair, B. Zoph, B. Ghor-
bani, B. Rossen, B. Sokolowsky, B. Barak, B. McGrew, B. Minaiev, B. Hao, B. Baker,
B. Houghton, B. McKinzie, B. Eastman, C. Lugaresi, C. Bassin, C. Hudson, C. M. Li,
C. de Bourcy, C. Voss, C. Shen, C. Zhang, C. Koch, C. Orsinger, C. Hesse, C. Fischer,
C. Chan, D. Roberts, D. Kappler, D. Levy, D. Selsam, D. Dohan, D. Farhi, D. Mely,
D. Robinson, D. Tsipras, D. Li, D. Oprica, E. Freeman, E. Zhang, E. Wong, E. Proehl,
E. Cheung, E. Mitchell, E. Wallace, E. Ritter, E. Mays, F. Wang, F. P. Such, F. Raso,
F. Leoni, F. Tsimpourlas, F. Song, F. von Lohmann, F. Sulit, G. Salmon, G. Parascandolo,
G. Chabot, G. Zhao, G. Brockman, G. Leclerc, H. Salman, H. Bao, H. Sheng, H. An-
drin, H. Bagherinezhad, H. Ren, H. Lightman, H. W. Chung, I. Kivlichan, I. O’Connell,
I. Osband, I. C. Gilaberte, I. Akkaya, I. Kostrikov, I. Sutskever, I. Kofman, J. Pachocki,
J. Lennon, J. Wei, J. Harb, J. Twore, J. Feng, J. Yu, J. Weng, J. Tang, J. Yu, J. Q.
Candela, J. Palermo, J. Parish, J. Heidecke, J. Hallman, J. Rizzo, J. Gordon, J. Uesato,
J. Ward, J. Huizinga, J. Wang, K. Chen, K. Xiao, K. Singhal, K. Nguyen, K. Cobbe,
K. Shi, K. Wood, K. Rimbach, K. Gu-Lemberg, K. Liu, K. Lu, K. Stone, K. Yu, L. Ah-
mad, L. Yang, L. Liu, L. Maksin, L. Ho, L. Fedus, L. Weng, L. Li, L. McCallum, L. Held,
L. Kuhn, L. Kondraciuk, L. Kaiser, L. Metz, M. Boyd, M. Trebacz, M. Joglekar, M. Chen,
M. Tintor, M. Meyer, M. Jones, M. Kaufer, M. Schwarzer, M. Shah, M. Yatbaz, M. Y.
Guan, M. Xu, M. Yan, M. Glaese, M. Chen, M. Lampe, M. Malek, M. Wang, M. Fradin,
M. McClay, M. Pavlov, M. Wang, M. Wang, M. Murati, M. Bavarian, M. Rohaninejad,
N. McAleese, N. Chowdhury, N. Chowdhury, N. Ryder, N. Tezak, N. Brown, O. Nachum,
O. Boiko, O. Murk, O. Watkins, P. Chao, P. Ashbourne, P. Izmailov, P. Zhokhov, R. Dias,
R. Arora, R. Lin, R. G. Lopes, R. Gaon, R. Miyara, R. Leike, R. Hwang, R. Garg, R. Brown,
R. James, R. Shu, R. Cheu, R. Greene, S. Jain, S. Altman, S. Toizer, S. Toyer, S. Mis-
erendino, S. Agarwal, S. Hernandez, S. Baker, S. McKinney, S. Yan, S. Zhao, S. Hu,
S. Santurkar, S. R. Chaudhuri, S. Zhang, S. Fu, S. Papay, S. Lin, S. Balaji, S. Sanjeev,
S. Sidor, T. Broda, A. Clark, T. Wang, T. Gordon, T. Sanders, T. Patwardhan, T. Sottiaux,
T. Degry, T. Dimson, T. Zheng, T. Garipov, T. Stasi, T. Bansal, T. Creech, T. Peterson,

- T. Eloundou, V. Qi, V. Kosaraju, V. Monaco, V. Pong, V. Fomenko, W. Zheng, W. Zhou, W. McCabe, W. Zaremba, Y. Dubois, Y. Lu, Y. Chen, Y. Cha, Y. Bai, Y. He, Y. Zhang, Y. Wang, Z. Shao, and Z. Li, “Openai o1 system card,” 2024.
- [2] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” 2025.
- [3] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askeel, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” 2022.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.
- [5] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” 2024.
- [6] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, J. Zhou, and J. Lin, “Group sequence policy optimization,” 2025.
- [7] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang, “Dapo: An open-source llm reinforcement learning system at scale,” 2025.
- [8] J. Hu, J. K. Liu, H. Xu, and W. Shen, “Reinforce++: Stabilizing critic-free policy optimization with global advantage normalization,” 2025.
- [9] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin, “Understanding r1-zero-like training: A critical perspective,” 2025.
- [10] M. Liu, S. Diao, X. Lu, J. Hu, X. Dong, Y. Choi, J. Kautz, and Y. Dong, “Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models,” 2025.

- [11] The OLMo Team, “Olmo 3,” tech. rep., Allen Institute for AI, November 2025. Technical Report.
- [12] MiniMax, :, A. Chen, A. Li, B. Gong, B. Jiang, B. Fei, B. Yang, B. Shan, C. Yu, C. Wang, C. Zhu, C. Xiao, C. Du, C. Zhang, C. Qiao, C. Zhang, C. Du, C. Guo, D. Chen, D. Ding, D. Sun, D. Li, E. Jiao, H. Zhou, H. Zhang, H. Ding, H. Sun, H. Feng, H. Cai, H. Zhu, J. Sun, J. Zhuang, J. Cai, J. Song, J. Zhu, J. Li, J. Tian, J. Liu, J. Xu, J. Yan, J. Liu, J. He, K. Feng, K. Yang, K. Xiao, L. Han, L. Wang, L. Yu, L. Feng, L. Li, L. Zheng, L. Du, L. Yang, L. Zeng, M. Yu, M. Tao, M. Chi, M. Zhang, M. Lin, N. Hu, N. Di, P. Gao, P. Li, P. Zhao, Q. Ren, Q. Xu, Q. Li, Q. Wang, R. Tian, R. Leng, S. Chen, S. Chen, S. Shi, S. Weng, S. Guan, S. Yu, S. Li, S. Zhu, T. Li, T. Cai, T. Liang, W. Cheng, W. Kong, W. Li, X. Chen, X. Song, X. Luo, X. Su, X. Li, X. Han, X. Hou, X. Lu, X. Zou, X. Shen, Y. Gong, Y. Ma, Y. Wang, Y. Shi, Y. Zhong, Y. Duan, Y. Fu, Y. Hu, Y. Gao, Y. Fan, Y. Yang, Y. Li, Y. Hu, Y. Huang, Y. Li, Y. Xu, Y. Mao, Y. Shi, Y. Wenren, Z. Li, Z. Li, Z. Tian, Z. Zhu, Z. Fan, Z. Wu, Z. Xu, Z. Yu, Z. Lyu, Z. Jiang, Z. Gao, Z. Wu, Z. Song, and Z. Sun, “Minimax-m1: Scaling test-time compute efficiently with lightning attention,” 2025.
- [13] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, “Alfworld: Aligning text and embodied environments for interactive learning,” 2021.
- [14] S. Yao, H. Chen, J. Yang, and K. Narasimhan, “Webshop: Towards scalable real-world web interaction with grounded language agents,” 2023.
- [15] Mistral-AI, :, A. Rastogi, A. Q. Jiang, A. Lo, G. Berrada, G. Lample, J. Rute, J. Barmentlo, K. Yadav, K. Khandelwal, K. R. Chandu, L. Blier, L. Saulnier, M. Dinot, M. Darrin, N. Gupta, R. Soletskyi, S. Vaze, T. L. Scao, Y. Wang, A. Yang, A. H. Liu, A. Sablayrolles, A. Héliou, A. Martin, A. Ehrenberg, A. Agarwal, A. Roux, A. Darcet, A. Mensch, B. Bout, B. Rozière, B. D. Monicault, C. Bamford, C. Wallenwein, C. Renaudin, C. Lanfranchi, D. Dabert, D. Mizelle, D. de las Casas, E. Chane-Sane, E. Fugier, E. B. Hanna, G. Delerce, G. Guinet, G. Novikov, G. Martin, H. Jaju, J. Ludziewski, J.-H. Chabran, J.-M. Delignon, J. Studnia, J. Amar, J. S. Roberts, J. Denize, K. Saxena, K. Jain, L. Zhao, L. Martin, L. Gao, L. R. Lavaud, M. Pellat, M. Guillaumin, M. Felardos, M. Augustin, M. Seznec, N. Raghuraman, O. Duchenne, P. Wang, P. von Platen, P. Saffer, P. Jacob, P. Wambergue, P. Kurylowicz, P. R. Muddireddy, P. Chagniot, P. Stock, P. Agrawal, R. Sauvestre, R. Delacourt, S. Gandhi, S. Subramanian, S. Dalal, S. Gandhi, S. Ghosh, S. Mishra, S. Aithal, S. Antoniak, T. Schueller, T. Lavril, T. Robert, T. Wang, T. Lacroix, V. Nemychnikova, V. Paltz, V. Richard, W.-D. Li, W. Marshall, X. Zhang, and Y. Tang, “Magistral,” 2025.
- [16] . Team, A. Zeng, X. Lv, Q. Zheng, Z. Hou, B. Chen, C. Xie, C. Wang, D. Yin, H. Zeng, J. Zhang, K. Wang, L. Zhong, M. Liu, R. Lu, S. Cao, X. Zhang, X. Huang, Y. Wei, Y. Cheng, Y. An, Y. Niu, Y. Wen, Y. Bai, Z. Du, Z. Wang, Z. Zhu, B. Zhang, B. Wen, B. Wu, B. Xu, C. Huang, C. Zhao, C. Cai, C. Yu, C. Li, C. Ge, C. Huang, C. Zhang, C. Xu, C. Zhu, C. Li, C. Yin, D. Lin, D. Yang, D. Jiang, D. Ai, E. Zhu, F. Wang, G. Pan, G. Wang, H. Sun, H. Li, H. Li, H. Hu, H. Zhang, H. Peng, H. Tai, H. Zhang, H. Wang, H. Yang, H. Liu, H. Zhao, H. Liu, H. Yan, H. Liu, H. Chen, J. Li, J. Zhao, J. Ren, J. Jiao, J. Zhao, J. Yan, J. Wang, J. Gui, J. Zhao, J. Liu, J. Li, J. Li, J. Lu, J. Wang, J. Yuan, J. Li, J. Du, J. Du, J. Liu, J. Zhi, J. Gao, K. Wang, L. Yang, L. Xu, L. Fan, L. Wu, L. Ding, L. Wang, M. Zhang, M. Li, M. Xu, M. Zhao, M. Zhai, P. Du, Q. Dong, S. Lei, S. Tu, S. Yang, S. Lu, S. Li, S. Li, Shuang-Li, S. Yang, S. Yi, T. Yu, W. Tian, W. Wang, W. Yu, W. L. Tam, W. Liang, W. Liu, X. Wang, X. Jia, X. Gu, X. Ling, X. Wang, X. Fan, X. Pan, X. Zhang, X. Zhang, X. Fu, X. Zhang, Y. Xu, Y. Wu, Y. Lu, Y. Wang, Y. Zhou,

- Y. Pan, Y. Zhang, Y. Wang, Y. Li, Y. Su, Y. Geng, Y. Zhu, Y. Yang, Y. Li, Y. Wu, Y. Li, Y. Liu, Y. Wang, Y. Li, Y. Zhang, Z. Liu, Z. Yang, Z. Zhou, Z. Qiao, Z. Feng, Z. Liu, Z. Zhang, Z. Wang, Z. Yao, Z. Wang, Z. Liu, Z. Chai, Z. Li, Z. Zhao, W. Chen, J. Zhai, B. Xu, M. Huang, H. Wang, J. Li, Y. Dong, and J. Tang, “Glm-4.5: Agentic, reasoning, and coding (arc) foundation models,” 2025.
- [17] A. Ahmadian, C. Cremer, M. Gallé, M. Fadaee, J. Kreutzer, O. Pietquin, A. Üstün, and S. Hooker, “Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms,” 2024.
- [18] Z. Liu, J. Liu, Y. He, W. Wang, J. Liu, L. Pan, X. Hu, S. Xiong, J. Huang, J. Hu, S. Huang, J. Obando-Ceron, S. Yang, J. Wang, W. Su, and B. Zheng, “Part i: Tricks or traps? a deep dive into rl for llm reasoning,” 2025.
- [19] D. Khatri, L. Madaan, R. Tiwari, R. Bansal, S. S. Duvvuri, M. Zaheer, I. S. Dhillon, D. Brandfonbrener, and R. Agarwal, “The art of scaling reinforcement learning compute for llms,” 2025.
- [20] K. Liu, J. K. Liu, M. Chen, and Y. Liu, “Rethinking kl regularization in rlhf: From value estimation to gradient optimization,” 2025.
- [21] Z. Xi, X. Guo, Y. Nan, E. Zhou, J. Shen, W. Chen, J. Liu, J. Huang, Z. Zhang, H. Guo, X. Deng, Z. Lei, M. Zheng, G. Wang, S. Zhang, P. Sun, R. Zheng, H. Yan, T. Gui, Q. Zhang, and X. Huang, “Bapo: Stabilizing off-policy reinforcement learning for llms via balanced policy optimization with adaptive clipping,” 2025.
- [22] G. Cui, Y. Zhang, J. Chen, L. Yuan, Z. Wang, Y. Zuo, H. Li, Y. Fan, H. Chen, W. Chen, Z. Liu, H. Peng, L. Bai, W. Ouyang, Y. Cheng, B. Zhou, and N. Ding, “The entropy mechanism of reinforcement learning for reasoning language models,” 2025.
- [23] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu, “Hybridflow: A flexible and efficient rlhf framework,” *arXiv preprint arXiv: 2409.19256*, 2024.
- [24] J. Hu, X. Wu, Z. Zhu, Xianyu, W. Wang, D. Zhang, and Y. Cao, “Openrlhf: An easy-to-use, scalable and high-performance rlhf framework,” *arXiv preprint arXiv:2405.11143*, 2024.
- [25] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with page-dattention,” in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [26] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: Efficient execution of structured language model programs,” 2024.
- [27] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li, “Pytorch fsdp: Experiences on scaling fully sharded data parallel,” 2023.
- [28] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” 2020.
- [29] F. Yao, L. Liu, D. Zhang, C. Dong, J. Shang, and J. Gao, “Your efficient rl framework secretly brings you off-policy rl training,” Aug. 2025.

- [30] H. He and T. M. Lab, “Defeating nondeterminism in llm inference,” *Thinking Machines Lab: Connectionism*, 2025. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- [31] Nemotron 3 Team, “Nvidia nemotron 3: Efficient and open intelligence,” tech. rep., NVIDIA, December 2025. Technical Report.
- [32] L. Feng, Z. Xue, T. Liu, and B. An, “Group-in-group policy optimization for llm agent training,” 2025.
- [33] Q. Team, “Qwen2.5: A party of foundation models,” September 2024.
- [34] Y. Qin, X. Tan, Z. He, G. Li, H. Lin, Z. Li, Z. Xu, Y. Shi, S. Cai, R. Rui, S. Cai, Y. Cai, X. Zhang, S. Ye, K. Li, and X. Sun, “Learn the ropes, then trust the wins: Self-imitation with progressive exploration for agentic reinforcement learning,” 2025.
- [35] W. Xu, W. Zhao, Z. Wang, Y.-J. Li, C. Jin, M. Jin, K. Mei, K. Wan, and D. N. Metaxas, “Epo: Entropy-regularized policy optimization for llm agents reinforcement learning,” 2025.
- [36] Z. Zhang, Z. Chen, M. Li, Z. Tu, and X. Li, “Rlvmr: Reinforcement learning with verifiable meta-reasoning rewards for robust long-horizon agents,” 2025.